

P48

LoRA: adaptación de rango bajo de modelos de lenguaje grandes

LoRA: Low-Rank Adaptation of Large Language Models

Edward J. Hu, Yelong Shen, Phillip Wallis, y otros · 2021 · arXiv:2106.09685 · ICLR 2022

Ficha pedagógica del Artificial Intelligence Evolution Program. No es el paper original: es una guía de lectura en español que enlaza a la fuente primaria. Consultado 2026-08-16.

P48 — LoRA

Arquitectura y entrenamiento · Si la actualización útil de un modelo tiene rango bajo, se puede entrenar factorizada — y ajustar un modelo grande deja de exigir un centro de datos.

Nivel: L3 · Motor: `lora` · Notebook: `P48_lora.ipynb` · Anexo: álgebra y geometría

1. Identificación

Campo	Valor
Título original	<i>LoRA: Low-Rank Adaptation of Large Language Models</i>
Autoría	Edward J. Hu, Yelong Shen, Phillip Wallis y otros
Año	2021
Venue	arXiv:2106.09685 · ICLR 2022
Fuente primaria	arXiv:2106.09685
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Ajustar un modelo grande a una tarea significaba actualizar **todos** sus pesos. Con GPT-3 eso son 175 000 millones de parámetros: memoria de optimizador prohibitiva —Adam guarda dos estados por parámetro—, y una copia completa del modelo por cada tarea.

Existían alternativas de ajuste parcial. Los *adapters* insertaban capas nuevas, pero añadían latencia en inferencia. El *prefix tuning* consumía parte de la ventana de contexto. Ninguna era gratis.

3. Propuesta

Congelar `W` y aprender su **actualización** en forma factorizada:

$$W' = W + BA, \quad \text{con } B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times d}, \quad r \ll d$$

La hipótesis, sugerida por trabajo previo sobre la dimensionalidad intrínseca del ajuste: la actualización que hace falta para adaptar el modelo a una tarea tiene **rango bajo**. Si es así, `BA` la representa con una fracción diminuta de los parámetros.

Y la propiedad que lo hace práctico: al desplegar, `BA` se **suma** a `W`. El modelo resultante es una matriz normal, sin capas extra. **Cero latencia añadida** en inferencia — a diferencia de los adapters.

4. Intuición sin fórmulas

Un manual de mil páginas y una fe de erratas de dos. No reimprimos el manual: publicas la fe de erratas, y cada especialidad tiene la suya sobre el mismo manual.

Dónde deja de funcionar la analogía: una fe de erratas corrige puntos sueltos. **BA** es una corrección que toca **toda** la matriz, pero con una estructura muy restringida —solo **r** direcciones independientes—. Es densa y de rango bajo, no dispersa.

5. Matemática mínima

Parámetros a entrenar:
ajuste completo : $d \times d$
LoRA : $2 \times d \times r$

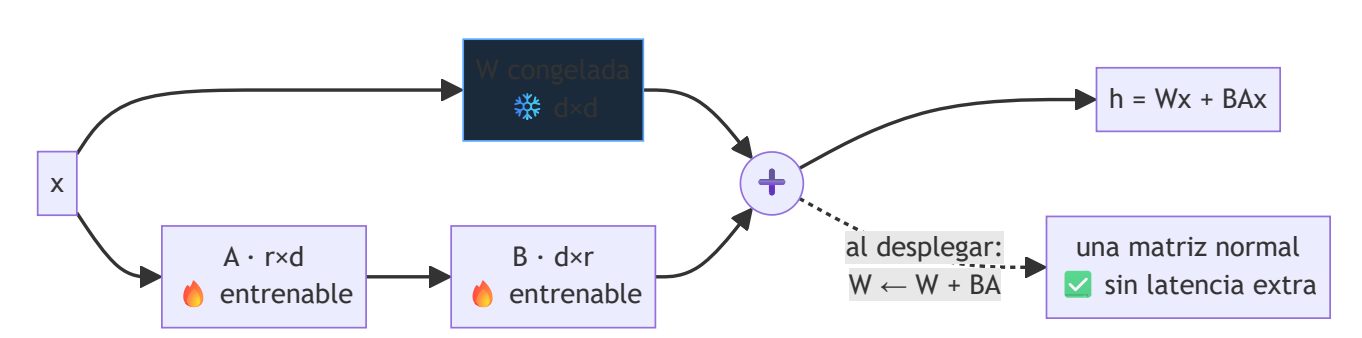
Con $d = 128$:

rango r	ajuste completo	LoRA	fracción	reducción
1	16 384	256	1,6 %	64×
4	16 384	1 024	6,3 %	16×
16	16 384	4 096	25 %	4×
64	16 384	16 384	100 %	1×

Con $r = d/2$ ya no se ahorra nada: $2 \cdot d \cdot r = d^2$. El método solo tiene sentido con **r** pequeño, y eso es una **apuesta** sobre la estructura del problema, no un teorema.

A se inicializa aleatoria y **B** a cero, de modo que $BA = 0$ al empezar: el ajuste arranca exactamente desde el modelo base.

6. Arquitectura o flujo



7. Qué observar en el paper original

- El **estudio de rango**: $r = 1$ o 2 ya funciona sorprendentemente bien en muchas tareas. Es la evidencia principal de la hipótesis de rango bajo.

- **A qué matrices aplicarlo:** los autores encuentran que las proyecciones de consulta y valor de la atención dan mejor resultado que repartir el presupuesto entre todas.
- La comparación explícita de **latencia** frente a adapters. Es el argumento de ingeniería que explica su adopción.
- El análisis de la **relación entre el subespacio de BA y las direcciones de W** , en el apéndice: sugiere que amplifica direcciones que ya existían pero estaban poco pesadas.

8. Evidencia y resultados

Experimentos sobre modelos de lenguaje de distintos tamaños en tareas de comprensión y generación, comparando ajuste completo, adapters, prefix tuning y LoRA a varios rangos.

Las cifras por tarea y rango están en el artículo. Verificarlas allí. Lo que hay que retener: con una fracción muy pequeña de parámetros entrenables, la calidad se mantiene comparable al ajuste completo en las tareas evaluadas.

La miniatura de este eje solo cuenta parámetros y enseña la forma de una actualización de rango 2. No entrena nada.

9. Impacto

- Hizo el ajuste de modelos grandes accesible fuera de los grandes laboratorios: un adaptador cabe en unos megabytes y se entrena en una GPU de consumo.
- Creó un **ecosistema**: miles de adaptadores compartidos sobre unos pocos modelos base, con servidores que cargan y descargan adaptadores por petición.
- Es la base de QLoRA, que lo combina con cuantización para bajar aún más el requisito de memoria.
- Y cambió la economía del ajuste fino: de decisión estratégica cara a experimento barato.

10. Limitaciones

1. **La hipótesis de rango bajo es empírica:** nada garantiza que la actualización útil lo sea para toda tarea, especialmente si exige conocimiento realmente nuevo.
2. **r y las matrices objetivo son hiperparámetros** cuya elección importa y no es automática.
3. **Menor capacidad de cambio que el ajuste completo:** con desplazamientos de dominio grandes, la diferencia aparece.
4. **Componer varios adaptadores no es trivial:** sumarlos no equivale a aprenderlos juntos.
5. **No reduce la memoria del modelo base** durante el entrenamiento —solo la del optimizador y los gradientes—; para eso hace falta cuantizar.
6. **La comparación con ajuste completo depende de la tarea y del presupuesto** del estudio.

11. Errores comunes

Error	Corrección
«Entrena menos parámetros, luego es más rápido»	Reduce memoria de optimizador y de gradientes, pero la pasada hacia delante sigue recorriendo el modelo entero.
«Añade latencia como los adapters»	No: BA se suma a W al desplegar y queda una matriz normal. Ese es su argumento central.
«Rango mayor siempre mejor»	El paper muestra que rangos muy pequeños bastan. Con $r = d/2$ se pierde todo el ahorro.
«Puede aprender cualquier cosa que aprenda el ajuste completo»	Está restringido a un subespacio de rango r . La hipótesis es empírica, no una garantía.
«Reduce la memoria necesaria para cargar el modelo»	La base sigue completa en memoria. Reducirla es lo que aporta QLoRA cuantizándola.

12. Relación con trabajos anteriores

- **P08 Transformer (2017)** — las matrices de proyección sobre las que se aplica.
- **P10 GPT-3 (2020)** — la escala que hace inviable el ajuste completo.
- **Adapters (Houlsby et al., 2019)** — la alternativa con coste de latencia. [arXiv:1902.00751](#)
- **Aghajanyan et al. (2020)** — la dimensionalidad intrínseca del ajuste, que motiva la hipótesis. [arXiv:2012.13255](#)

13. Relación con trabajos posteriores

- **P49 QLoRA (2023)** — LoRA sobre una base cuantizada a 4 bits.
- **DoRA, LoRA+, AdaLoRA (2023-2024)** — variantes que reparten el rango o descomponen magnitud y dirección.
- **Servidores multiadaptador (2023+)** — infraestructura que explota que la base es compartida.
- **P45 Destilación (2015)** — la vía alternativa para abaratar: achicar el modelo en vez de su ajuste.

14. Notebook asociado

`P48_lora.ipynb`

Qué implementa: el conteo de parámetros para cuatro rangos con $d = 128$, y la construcción explícita de una actualización de rango 2 como producto BA .

Qué NO implementa: no entrena nada. Ni modelo, ni tarea, ni comparación de calidad. La hipótesis central del paper —que la actualización útil es de rango bajo— aquí se enuncia, no se verifica.

```
ai-evolution paper-lab P48 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la ecuación de LoRA e identifica qué se congela.
Explicar	Explica por qué no añade latencia en inferencia.
Aplicar	Calcula el ahorro con $d = 4096$ y $r = 8$.
Analizar	¿A partir de qué r deja de haber ahorro y por qué?
Evaluar	Una tarea exige conocimiento de dominio muy nuevo. ¿Confiarías en LoRA?
Crear	Diseña un experimento que mida si la actualización útil de una tarea es de rango bajo.

16. Autoevaluación

1. ¿Qué se entrena y qué se congela?
2. ¿Por qué B se inicializa a cero?
3. ¿Por qué no hay latencia añadida en inferencia?
4. ¿Qué memoria ahorra exactamente y cuál no?
5. ¿A partir de qué rango desaparece el ahorro?
6. ¿Es la hipótesis de rango bajo un teorema?
7. ¿Qué añade QLoRA sobre esto?

17. Respuestas esperadas

1. Se entrenan las dos matrices pequeñas A y B ; la matriz original W queda congelada y no recibe gradiente.
2. Para que $BA = 0$ al inicio y el modelo arranque exactamente en el comportamiento del modelo base, sin una perturbación aleatoria de partida.
3. Porque al desplegar se calcula $W + BA$ una sola vez y queda una matriz de las mismas dimensiones. No hay capas adicionales que recorrer.
4. Ahorra la memoria del optimizador y de los gradientes, que solo se necesitan para los parámetros entrenables. **No** ahorra la memoria de cargar el modelo base, que sigue completo.
5. A partir de $r = d/2$, donde $2 \cdot d \cdot r = d^2$ y se entrenan tantos parámetros como en el ajuste completo.
6. No. Es una hipótesis empírica bien respaldada en las tareas evaluadas, no una garantía general.
7. Cuantizar la base congelada a 4 bits, que es justo la memoria que LoRA no reduce.

18. Fuentes primarias

- Hu, E. J. et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models*. **ICLR 2022**. arXiv:2106.09685 · consultado 2026-08-16.
- Aghajanyan, A., Zettlemoyer, L. y Gupta, S. (2020). *Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning*. arXiv:2012.13255 · consultado 2026-08-16.



Evaluación — P48 · LoRA: adaptación de rango bajo de modelos de lenguaje grandes

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *LoRA: Low-Rank Adaptation of Large Language Models* (2021, arXiv:2106.09685 · ICLR 2022) ·

Nivel: L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P48_lora.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).